

Méthode des tableaux: Applications dans différentes logiques

TIPE 25/26 - Cycles et Boucles

GIL Dorian - Candidat n°20220

La Méthode des tableaux (Logique Propositionnelle)

Algorithme de résolution du problème SAT

La Méthode des tableaux (Logique Propositionnelle)

Algorithme de résolution du problème SAT

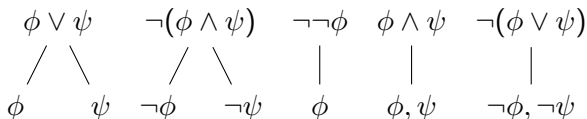
Règle d'inférence :

$$\begin{array}{ccccc} \phi \vee \psi & \neg(\phi \wedge \psi) & \neg\neg\phi & \phi \wedge \psi & \neg(\phi \vee \psi) \\ \swarrow \quad \searrow & \swarrow \quad \searrow & | & | & | \\ \phi & \neg\phi & \phi & \phi, \psi & \neg\phi, \neg\psi \end{array}$$

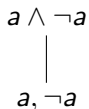
La Méthode des tableaux (Logique Propositionnelle)

Algorithme de résolution du problème SAT

Règle d'inférence :



Exemple avec $a \wedge \neg a$:



Definition (Forme Alternée)

Soit $n \in \mathbb{N}^*$, et $(a_k)_{k \in [1, n]}$ des littéraux, on dit que φ est de forme alternée ssi $\varphi = a_1 \wedge (a_2 \vee (a_3 \wedge (\dots (a_n))))$

Definition (Forme Alternée)

Soit $n \in \mathbb{N}^*$, et $(a_k)_{k \in [1, n]}$ des littéraux, on dit que φ est de forme alternée ssi $\varphi = a_1 \wedge (a_2 \vee (a_3 \wedge (\dots (a_n))))$

Objectif de l'étude introductive ?

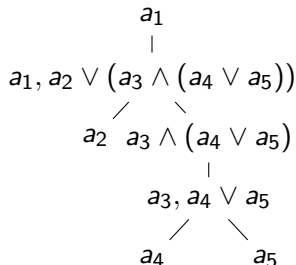
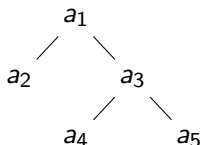
Etude introductive

Definition (Forme Alternée)

Soit $n \in \mathbb{N}^*$, et $(a_k)_{k \in [1, n]}$ des littéraux, on dit que φ est de forme alternée ssi $\varphi = a_1 \wedge (a_2 \vee (a_3 \wedge (\dots (a_n))))$

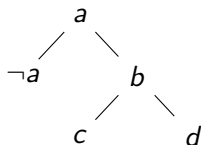
Objectif de l'étude introductive ?

Une re-écriture :



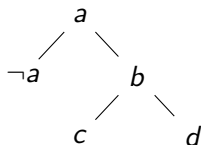
Proposition d'algorithme ($\mathcal{O}(n)$, correction prouvé) :

- 1 Fin de l'arbre : Renvoyer Vrai
- 2 Feuille gauche : SI Pas Contradiction, Renvoyer Vrai
- 3 Feuille droite : SI Contradiction, Renvoyer Faux SINON Appel
Recuratif



Proposition d'algorithme ($\mathcal{O}(n)$, correction prouvé) :

- 1 Fin de l'arbre : Renvoyer Vrai
- 2 Feuille gauche : SI Pas Contradiction, Renvoyer Vrai
- 3 Feuille droite : SI Contradiction, Renvoyer Faux SINON Appel Recursif



Pour 100 Formes alternées, temps d'exécution :

- Alternée : 0.000493s
- Quine (avec conversion en CNF) : 0.025874s
- Quine (sans conversion en CNF) : 0.018691s

Conclusion

Quelques Remarques

- Conversion IMPOSSIBLE

Quelques Remarques

- Conversion IMPOSSIBLE
- Méthode des tableaux en logique propositionnelle ?

Quelques Remarques

- Conversion IMPOSSIBLE
- Méthode des tableaux en logique propositionnelle ?
- Réelle application de la méthode ?

Definitions 1/2

Definition (Formule de la LTL)

On définit F l'ensemble des formules de la LTL inductivement par :

- $p \in AP \implies p \in F$
- si ψ et ϕ sont des formules de LTL alors
 $\neg\psi, \phi \vee \psi, X\psi, \phi U \psi, F\psi, G\psi$ sont des formules de LTL

AP un ensemble fini de variables propositionnelles.

Definitions 1/2

Definition (Formule de la LTL)

On définit F l'ensemble des formules de la LTL inductivement par :

- $p \in AP \implies p \in F$
- si ψ et ϕ sont des formules de LTL alors
 $\neg\psi, \phi \vee \psi, X\psi, \phi U \psi, F\psi, G\psi$ sont des formules de LTL

AP un ensemble fini de variables propositionnelles.

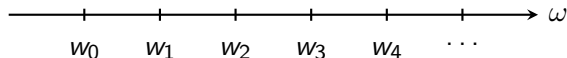
Definition (Opérateurs X, U, F, G)

- $X\phi$: ϕ doit être satisfaite dans l'état suivant (neXt)
- $\psi U \phi$: ψ doit être satisfaite dans tous les états jusqu'à un état où ϕ est satisfait (Until)
- $F\phi$: ϕ doit être satisfaite une fois plus tard (Finally)
- $G\phi$: ϕ doit être satisfaite dans tous les états futurs (Globally)

Definitions 2/2

Definition (Valuation)

On définit un tel objet comme $\omega := w_0, w_1, \dots$ une suite infinie de monde.



Pourquoi ?

Pourquoi la LTL ?

- Preuve de programme concurrentiel
- Raisonnement sur des circuits intégrés
- Raisonnement sur les protocoles de communications

Pourquoi ?

Pourquoi la LTL ?

- Preuve de programme concurrentiel
- Raisonnement sur des circuits intégrés
- Raisonnement sur les protocoles de communications

Definition (Verification de modèle (Model Checking))

Technique automatique qui étant donné un modèle à nombre d'état fini d'un système et des propriétés formelles, vérifie systématiquement si ces propriétés restent vrai pour ce modèle.
(Principles of Model Checking de C.Baier, JP.Katoen)

La méthode des tableaux dans tout cela ?

- Utiliser dans les algorithmes de vérification de modèle
- Problème de ces algorithmes

La méthode des tableaux dans tout cela ?

- Utiliser dans les algorithmes de vérification de modèle
- Problème de ces algorithmes

Etude intéressante à faire : Comment modéliser à partir de la LTL un système et ces contraintes ?

La méthode des tableaux dans tout cela ?

- Utiliser dans les algorithmes de vérification de modèle
- Problème de ces algorithmes

Etude intéressante à faire : Comment modéliser à partir de la LTL un système et ces contraintes ? Pour répondre à cela :

- On va prendre un exemple de système et le modéliser
- On comparera ce qu'on a produit à la littérature

Définition du système

Compteur Binaire Modulo 4 : Tout les quatres cycles, il renvoie 1 (vrai) sinon 0 (faux).

Définition du système

Compteur Binaire Modulo 4 : Tout les quatres cycles, il renvoie 1 (vrai) sinon 0 (faux).

Hypothèse : Le compteur est fait par des circuits logiques (on a 2 ampoules) On notera formellement les ampoules φ_1 , φ_2 et la valeur renvoyé r

Propriétés

- L'ampoule 2 change toujours de valeur
- L'ampoule 1 change ssi φ_2
- r ssi les deux ampoules sont éteintes

Propriétés

- L'ampoule 2 change toujours de valeur
- L'ampoule 1 change ssi φ_2
- r ssi les deux ampoules sont éteintes

Traduction

- $G(\varphi_2 \Leftrightarrow X(\neg\varphi_2))$
- $G(\varphi_2 \Leftrightarrow (\varphi_1 \Leftrightarrow X(\neg\varphi_1)))$
- $G(r \Leftrightarrow \neg(\varphi_1 \vee \varphi_2))$

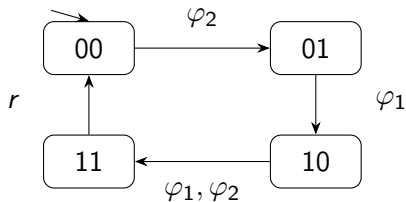
Propriétés

- L'ampoule 2 change toujours de valeur
- L'ampoule 1 change ssi φ_2
- r ssi les deux ampoules sont éteintes

Traduction

- $G(\varphi_2 \Leftrightarrow X(\neg\varphi_2))$
- $G(\varphi_2 \Leftrightarrow (\varphi_1 \Leftrightarrow X(\neg\varphi_1)))$
- $G(r \Leftrightarrow \neg(\varphi_1 \vee \varphi_2))$

Remarque : G est essentiel



Proposition de système de transition

Comparaison

Source : Principles of Model Checking de C.Baier, JP.Katoen

- Même système de transition
- $G(r \Leftrightarrow \neg\varphi_1 \wedge \neg\varphi_2)$ similaire
- $G(\varphi_1 \implies (Xr \vee XXr))$
- $G(r \implies (X\neg r \wedge XX\neg r))$
- $G(r \vee Xr \vee XXr \vee XXXr)$
- $G(r \implies (X\neg r \wedge XX\neg r \wedge XXX\neg r))$

Comparaison

Source : Principles of Model Checking de C.Baier, JP.Katoen

- Même système de transition
- $G(r \Leftrightarrow \neg\varphi_1 \wedge \neg\varphi_2)$ similaire
- $G(\varphi_1 \implies (Xr \vee XXr))$
- $G(r \implies (X\neg r \wedge XX\neg r))$
- $G(r \vee Xr \vee XXr \vee XXXr)$
- $G(r \implies (X\neg r \wedge XX\neg r \wedge XXX\neg r))$

Remarques :

- A priori, cycle modulo 4 pas assuré par mes contraintes
- Leur contrainte laisse libre l'ordre d'allumage des lumières
- Ce n'est pas une preuve de correction

Conclusion

Rappel : On voulait déterminer l'utilité de la méthode des tableaux dans différentes logiques.

- Etude de la résolution de SAT et familiarisation avec la méthode
- Application et discussion à propos des algorithmes utilisant cette méthode

```

1 type formula = (* Definition d'un type simple de
    ↪ formule logique *)
2   | Atom of (string* bool)
3   | And of (string*bool) * formula
4   | Or of (string*bool) * formula
5
6 type branch = (* Definition de notre stockage des
    ↪ branches*)
7   | Empty
8   | Node of (formula option * formula * branch);;
9
10 let extract (f:formula option) = match f with
11   | None -> Atom("none", false)
12   | Some t -> t
13
14 let rec print_formula (f:formula) = match f with

```

```

15 | Atom(s, b) -> if b then print_string s else
    ↪ print_string "Not ";print_string s;
16 | And ((f, b),g) -> if b then print_string f
    ↪ else print_string "Not ";print_string
    ↪ f;print_string " And ";print_formula g
17 | Or ((f,b),g) -> if b then print_string f else
    ↪ print_string "Not ";print_string
    ↪ f;print_string " Or ";print_formula g;;
18
19 let rec print_branches (b:branch) =
20   print_string " [";
21   match b with
22   | Empty -> ()
23   | Node(a1, a2, b) -> print_formula@@extract
    ↪ a1;print_string ", ";print_formula
    ↪ a2;print_branches b;

```

```

24   print_string "]";;
25
26   (* Construction du tableau en  $O(\text{nb\_connecteur ET})$ 
    ↪ *)
27   let rec formula2branch (f:formula) : branch =
    ↪   match f with
28     | And(a, Or(b, Atom(c))) -> Node(Some(Atom b),
    ↪   Atom a, Node(None, Atom(c), Empty))
29     | And(a, Or(b, c)) -> Node(Some(Atom b), Atom
    ↪   a, formula2branch c)
30     | And(a, Atom(b)) -> Node(Some (Atom b), Atom a,
    ↪   Empty)
31     | _ -> failwith "Pas alternée"
32
33   let has_cycle (br:branch) : bool =

```

```

34  let rec aux (br:branch) (d:(string,bool)
    ↪  Hashtbl.t) : bool = match br with
35  | Node(None, Atom (f, b), Empty) ->
36      if Hashtbl.mem d f then
37          Hashtbl.find d f = b
38      else
39          true
40  | Node(Some(Atom(fg, bg)), Atom (fd, bd), Empty)
    ↪  ->
41      if Hashtbl.mem d fd then
42          if Hashtbl.find d fd = bd then
43              not @@ (Hashtbl.mem d fg &&
                     ↪  Hashtbl.find d fg <> bg)
44          else
45              false
46      else(

```

```

47         Hashtbl.add d fd bd;
48         not @@ (Hashtbl.mem d fg && Hashtbl.find
49             ↪ d fg <> bg)
50     )
51 | Node(Some (Atom (fg, bg)), Atom (fd, bd), nb)
52 ↪ ->
53     if Hashtbl.mem d fd then
54         if Hashtbl.find d fd <> bd then
55             false
56         else
57             if Hashtbl.mem d fg then
58                 if Hashtbl.find d fg = bg then
59                     true
60                 else
61                     aux nb d
62             else

```

```

61         true
62     else
63         (Hashtbl.add d fd bd;
64         if Hashtbl.mem d fg then
65             if Hashtbl.find d fg = bg then
66                 true
67             else
68                 aux nb d
69         else
70             true)
71     | _ -> failwith "Pas alternée"
72 in aux br (Hashtbl.create 100);;
73
74 let is_satisfiable (f:formula) : bool = let b =
75     ↪ formula2branch f in has_cycle b;;

```

```
1 from random import randint, getrandbits
2 from math import floor
3
4
5 def SAT_formule_gen(l):
6     string = "("
7     for i in range(l):
8         b = bool(getrandbits(1))
9         if i % 2 == 0:
10             string += f"And("
11         else:
12             string += f"Or("
13         if b:
14             string += f"Var {randint(0, 10)}),"
15     else:
```

```

16         string += f"Not(Var {randint(0,
           ↪ 10)}),"
17     if b:
18         string += f"Var {randint(0, 10)}"
19     else:
20         string += f"Not(Var {randint(0, 10)})"
21     string += (l+1)*")"
22     return string
23
24 def alternee_formule_gen(l, nb_lit):
25     string = "("
26     for i in range(l):
27         b = bool(getrandbits(1))
28         if i % 2 == 0:
29             string += f"And("
30         else:

```

```
31         string += f"Or("
32     if b:
33         string += f"(\",{randint(0, nb_lit)})\",
34             ↪ true),"
35     else:
36         string += f"(\",{randint(0, nb_lit)})\",
37             ↪ false),"
38 if b:
39     string += f"Atom(\",{randint(0, nb_lit)})\",
40         ↪ true)"
41 else:
42     string += f"Atom(\",{randint(0, nb_lit)})\",
43         ↪ false)"
44 string += (l+1)*")"
45 return string
```

```
43 def both_gen(l, nb_lit):
44     alt_str = "("
45     sat_str = "("
46     dpl_str = ""
47     for i in range(l):
48         b = bool(getrandbits(1))
49         if i % 2 == 0:
50             alt_str += f"And("
51             sat_str += f"And("
52         else:
53             alt_str += f"Or("
54             sat_str += f"Or("
55         number = randint(0, nb_lit)
56
57         if b:
58             alt_str += f"({number})", true),"
```

```
59         sat_str += f"Var {number},"
60         dpl_str += f"{number}"
61     else:
62         alt_str += f"(\\"{number}\\", false),"
63         sat_str += f"Not(Var {number}),"
64         dpl_str += f"-{number}"
65
66     if i % 2 == 0:
67         dpl_str += " & ("
68     else:
69         dpl_str += " | ("
70     number = randint(0, nb_lit)
71     if b:
72         alt_str += f"Atom(\\"{number}\\", true)"
73         sat_str += f"Var {number}"
74         dpl_str += f"{number}"
```

```
75     else:
76         alt_str += f"Atom(\"{number}\", false)"
77         sat_str += f"Not(Var {number})"
78         dpl_str += f"-{number}"
79     alt_str += (l+1)*")"
80     sat_str += (l+1)*")"
81     dpl_str += l*")"
82     return alt_str, sat_str, dpl_str
83
84 txt_alt = "["
85 txt_sat = "["
86 txt_dpl = ""
87 for i in range(1, 101):
88     res = both_gen(i, randint(0, floor(i/2)))
89     txt_alt += res[0]+";\n"
90     txt_sat += res[1]+";\n"
```

```

91     txt_dpl += res[2]+"\n"
92 txt_alt += "|"
93 txt_sat += "|"
94 with open("tests_alt.txt", "w") as f_alt:
95     f_alt.write(txt_alt)
96 with open("tests_quine.txt", "w") as f_sat:
97     f_sat.write(txt_sat)
98 with open("tests_dp11.txt", "w") as f_dp11:
99     f_dp11.write(txt_dpl)

1 type literal = V of int | NV of int;;
2 type clause = literal list;;
3 type cnf = clause list;;
4
5 type proposition =
6   | Var of int
7   | Not of proposition

```

```

8   | And of proposition * proposition
9   | Or of proposition * proposition
10
11 let rec nnf prop =
12   match prop with
13   | Var _ -> prop
14   | Not (Var _ as v) -> Not v
15   | Not (Not p) -> nnf p
16   | Not (And (p, q)) -> Or (nnf (Not p), nnf (Not
    ↪ q))
17   | Not (Or (p, q)) -> And (nnf (Not p), nnf (Not
    ↪ q))
18   | And (p, q) -> And (nnf p, nnf q)
19   | Or (p, q) -> Or (nnf p, nnf q)
20
21 let rec distribute_or p1 p2 =

```

```
22  match (p1, p2) with
23  | (And (a, b), x) ->
24      And (distribute_or a x, distribute_or b x)
25  | (x, And (a, b)) ->
26      And (distribute_or x a, distribute_or x b)
27  | _ -> Or (p1, p2)
28
29  let rec to_cnf prop =
30  match prop with
31  | And (p, q) -> And (to_cnf p, to_cnf q)
32  | Or (p, q) ->
33      let cp = to_cnf p in
34      let cq = to_cnf q in
35      distribute_or cp cq
36  | _ -> prop
37
```

```
38 let rec prop_to_cnf prop =
39   prop
40   |> nnf
41   |> to_cnf
42
43 let rec literal_of_prop p =
44   match p with
45   | Var n -> V n
46   | Not (Var n) -> NV n
47   | _ -> failwith "Invalid literal in clause"
48
49 let rec prop_to_cnf_data prop =
50   match prop with
51   | And (p, q) ->
52     (prop_to_cnf_data p) @ (prop_to_cnf_data q)
53   | _ ->
```

```

54      (* a clause is a disjunction of literals *)
55      let rec gather_literals p lits =
56          match p with
57          | Or (p1, p2) -> gather_literals p1
58              ↪ (gather_literals p2 lits)
59          | _ ->
60              let lit = literal_of_prop p in
61              lit :: lits
62          in
63          [gather_literals prop []]
64      let proposition_to_cnf p =
65          p |> prop_to_cnf |> prop_to_cnf_data
66
67      let litt_of_name_and_bool x b =
68          if b then V x else NV x;;

```

```
69
70 let printclause c=
71   Printf.printf "[";
72   let rec aux = function
73     [] -> Printf.printf "]";
74     | l::q -> match l with
75       | NV x -> Printf.printf "NV %d; " x; aux q;
76       | V x -> Printf.printf "V %d; " x; aux q;
77   in aux c
78 ;;
79
80 let printprop p = Printf.printf "prop = ";
81   let rec aux = function
82     [] -> Printf.printf "]\n";
83     | c::q -> printclause c; aux q;
84   in aux p;;
```

```

85
86 let delete_lit (c:clause) (x:int) (b:bool) =
87   List.filter
88     ((<>) @@ litt_of_name_and_bool x (not b))
89     c;;
90
91 let rec subst (f:cnf) (x:int) (b:bool) : cnf
92   ↪ option =
93   match f with
94   [] -> Some []
95   | c::q -> let q' = subst q x b in
96             match q' with
97             | None -> None
98             | Some q2 -> let c = delete_lit c x b
99                           ↪ in
100                           if c = [] then None else

```

```

99         if List.mem
           ↪ (litt_of_name_and_bool
           ↪ x b) c
100     then Some q2
101     else Some (c::q2);;
102
103 let get_var (f:cnf) : int = match f with
104   | (V x::_)::_ | (NV x::_)::_ -> x
105   | _ -> failwith "pas de variable";;
106
107 let rec quine f = match f with
108   | [] -> true
109   | _ -> let x = get_var f in
110         (*totest : substitution partielle*)
111         let rec aux f totest = match totest with

```

```

112     | [] -> false (*on a testé  $x \leftarrow T$  et  $x \leftarrow F$  sans
      ↪ succès*)
113     | b::q -> let sf = subst f x b (* $x \leftarrow b$ *)
114         in match sf with
115         | Some f2 ->
116             if quine f2
117             then true (*un choix de substitution
      ↪ donne true*)
118             else aux f q (* $x \leftarrow b$  marche pas, on
      ↪ essaye le suivant*)
119         | None -> aux f q
120     in aux f [true;false];;
121
122 let sat (f:proposition) : bool = quine @@
      ↪ proposition_to_cnf f;;
123

```

```
124 let tests (fa: proposition array): unit =
125   let t = Sys.time() in
126   let remove_t = ref 0. in
127   for i = 0 to Array.length fa -1 do
128     let t1 = Sys.time() in
129     let form = proposition_to_cnf fa.(i) in
130     remove_t := !remove_t +. (Sys.time() -. t1);
131     let _ = quine form in ()
132 done;Printf.printf "Execution time (QUINE):
    ↪ %fs\n" (Sys.time() -. t -. !remove_t);;
```